Graphs

CSC 3449

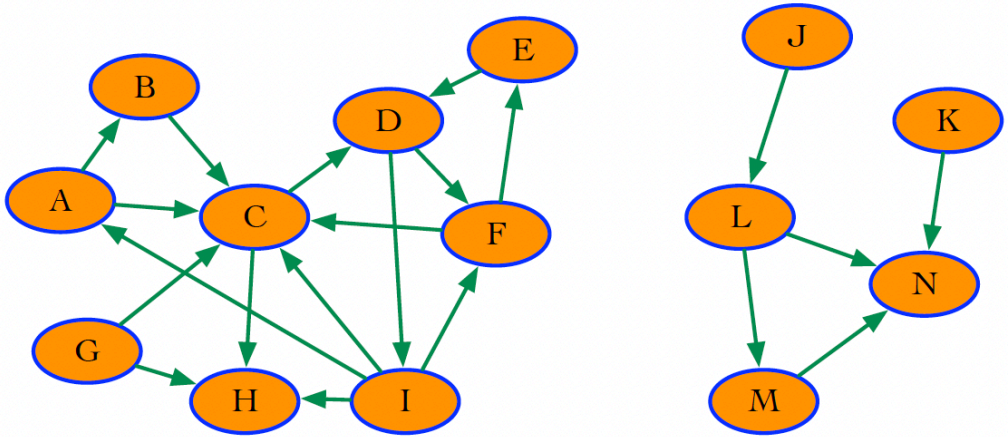
Rodrigo Canaan
Adapted with permission from Theresa t

Directed Graphs

While undirected graphs can model many real-world problems or systems, sometimes we need to represent one-directional relationships

- Some roads can only be driven in one direction
- Rivers flow downstream
- Most family relationships (parent, child, descendant, ancestor)
- “Follows” in social networks
- Food chains
- Hierarchical structures

Directed Graphs



- We now treat edges as *directed* pairs. Here, $(c, d) \in E$ but $(d, c) \notin E$
- If using an adjacency list, *v* may appear on *u's* list but not vice-versa
- If using an adjacency matrix, the matrix is no longer necessarily symmetrical.

Definition

The *in degree* of a vertex is the number of edges leading into it. The in degree of C is 5
The *out degree* is the number of edges leading out of it. The out degree o C is 2

DFS in Directed Graphs

Definition

A *path* in a directed graph is a sequence of distinct vertices where there is an edge between all pairs of successive vertices, and all edges point in the same direction
A vertex *u* is *reachable* from *v* if there is a path from *v* to *u*. There might not be a path from *u* to *v*

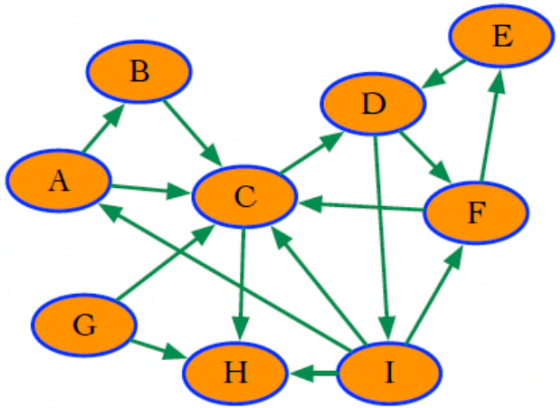
DFS can still determine reachability in directed graphs and works with a small change:

DFS-visit (G, v)

Input: A **directed** graph $G = \{V, E\}$, a vertex v. Initially, every vertex has discovered set to False
Output: Every vertex reachable from v is flagged as discovered
1. v.discovered ← True
2. For all **outgoing** neighbors u of v:
    2.1.if $u\,.\,discovered$ is False
        2.1.1.DFS-Visit(G, u)

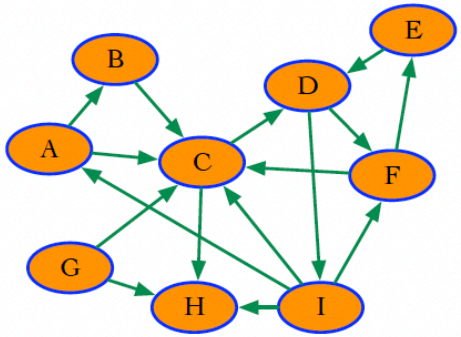
DFS in Directed Graphs

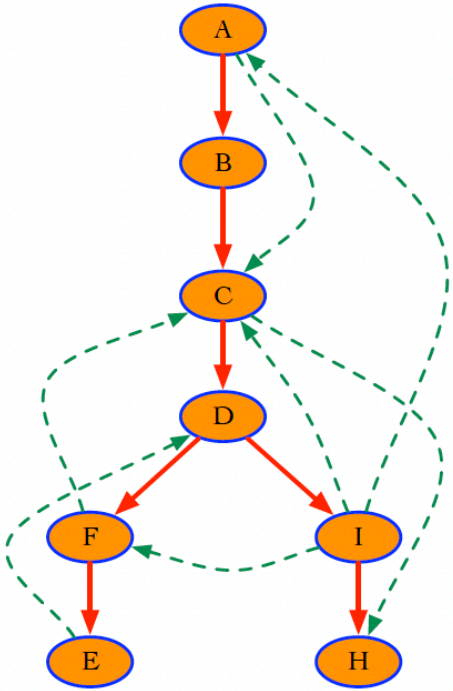
Example: perform DFS on the following directed graph



DFS in Directed Graphs

Example: perform DFS on the following directed graph



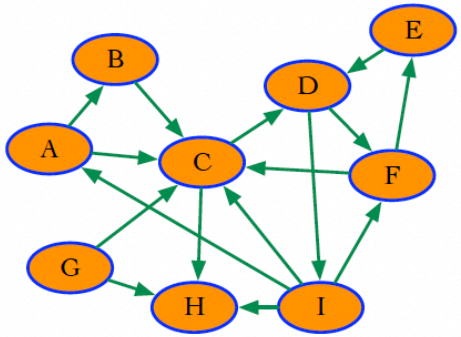


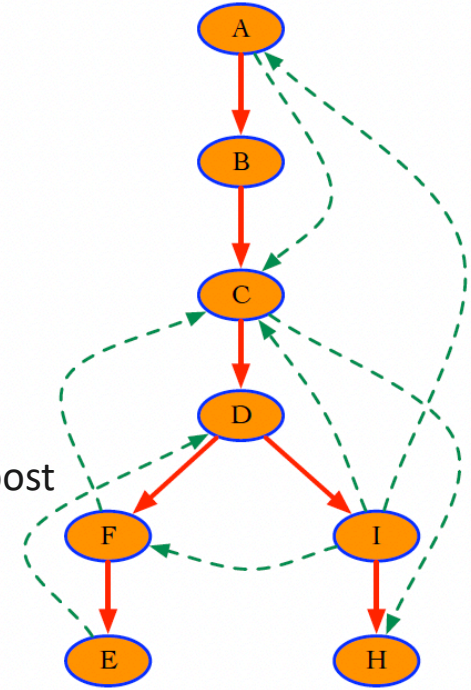
There are now *four* types of edges:

- Tree edges
- Forward edges: lead to a non child descendant
- Back edges: lead to an ancestor
- Cross edges: lead to neither an ancestor nor descendant

DFS in Directed Graphs

Example: perform DFS on the following directed graph



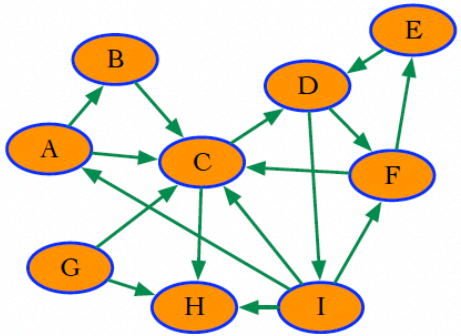


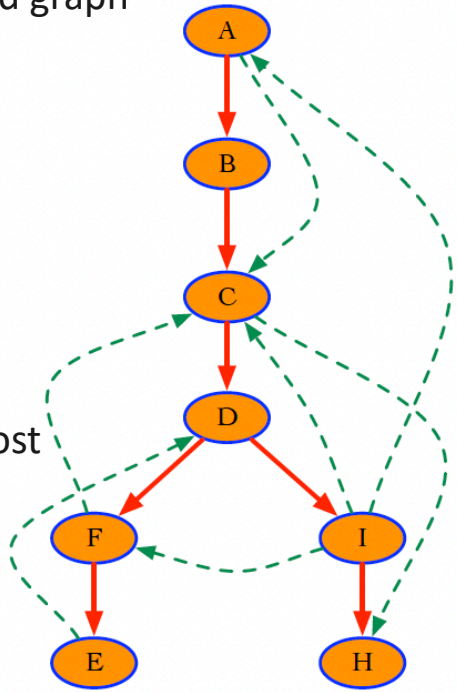
Edge types can also be characterized by pre-and post numbers. If an edge goes from u to v, and it is a

- Tree edges: pre(u)<pre(v)<post(v)<pre(u)
- Forward edges: same as tree edges
- Back edges: pre(v)<pre(u)<post(u)<post(v)
- Cross edges: pre(v)<post(v)<pre(u)<post(u)

DFS in Directed Graphs

Example: perform DFS on the following directed graph





Edge types can also be characterized by pre-and post numbers intervals. If an edge goes from **u** to **v**

- Tree edges: [[ ]]
- Forward edges: same as tree edges
- Back edges: [[ ]]
- Cross edges: [ ][ ]

Strong and Weak Components

Definition

A *Strongly Connected Component* (or *strong component*)in a directed graph is a maximal subset of its vertices for which there is a path between each pair of vertices.

This is the same definition as for undirected graphs, but the implications are very different:

- Reachability is no longer symmetrical
- The components of a graph no longer partition it into neatly separated “islands”

Definition

A *Weakly Connected Component* (or *weak component*) in a directed graph is a maximal subset of vertices for which, if you ignore edge directions, all pairs of vertices are mutually reachable

These correspond to the “islands”. Think of it as erasing all arrows from the edges.

Strong and Weak Components

Finding weak components is easy: just treat the graph as if it were undirected.

Finding strong components isn’t as simple: a single DFS pass only guarantees one-way reachability

- You could do it naively by checking reachability both ways for every pair of vertices.
    - Can you tell the time complexity of this approach?
- Doing it in linear time is possible, and is what we will build towards in the next few slides.

Cycles and DAGs

Definition

A *cycle* in a directed graph is a walk with **two** or more edges, that starts and ends on the same vertex, but otherwise all vertices are distinct.

Note: the cycle definition for undirected graphs requires three or more edges. Why?

Definition

A *Directed Acyclic Graph (DAG)* is a directed graph with no cycles

Analogous to trees in that it has no cycles, but some properties are lost:

- In a tree, there is a single path between any two vertices. In a DAG, there may be multiple
- A tree always has n-1 edges. In a DAG, you can have up to O(n^2) edges
- In a tree, any two vertices are mutually reachable. In a DAG, they are *never* mutually reachable (why?) and some pairs of vertices may not be reachable in either direction (even if the DAG is weakly connected)

Sources and Sinks

Definition

A *source* in a directed graph is a vertex with no incoming edges
A *sink* is a vertex with no outgoing edges.

Lemma: every finite DAG contains at least one source and at least one sink

Proof (sink):

- Start with any vertex and, if it is not a sink, follow an arbitrary edge.
- Iteratively do so until you find a sink or a repeat vertex.
- Because the graph is finite, one of these must eventually happen.
- But a repeat vertex would make a cycle, which is not possible in a DAG.
- Therefore, we must eventually find a sink.

The proof for sources is analogous, but we follow edges in reverse direction.

DAGs and Topological Sorting

A common use for DAGs is to model complex dependency relations.

For example, in scheduling, tasks can be vertices and pre-requisites can be edges.

- A task can have multiple pre-requisites.
- But a cycle would represent an irresolvable set of pre-requisites (A requires B, B requires C, C requires A)

A useful property of DAGs is that they can be *topologically sorted*

Definition

A *topological sort* of a DAG is a list of all its vertices (v1, v2, v3…) such that every edge in the DAG goes from a vertex with lower index to one with a higher index (all arrows point in the same direction.

In scheduling, this corresponds to a legal sequence of tasks (with no parallelism)

How can we topologically sort of a DAG?

DAGs and Topological Sorting

A common use for DAGs is to model complex dependency relations.

For example, in scheduling, tasks can be vertices and pre-requisites can be edges.

- A task can have multiple pre-requisites.
- But a cycle would represent an irresolvable set of pre-requisites (A requires B, B requires C, C requires A)

A useful property of DAGs is that they can be *topologically sorted*

Definition

A *topological sort* of a DAG is a list of all its vertices (v1, v2, v3…) such that every edge in the DAG goes from a vertex with lower index to one with a higher index (all arrows point in the same direction.

In scheduling, this corresponds to a legal sequence of tasks (with no parallelism)

How can we topologically sort of a DAG?

Answer: Perform DFS, then list vertices in reverse post order (or put vertices in a stack at post-visit

The structure of strong components

We can think of two types of edges involving the vertices in the component:

- Internal: Edges *within* the component (making one or more cycles).
- External: Edges from the component to the rest of the graph, or from the rest of the graph into the component

Consider “contracting” the component to a single “meta-vertex”, ignoring edges within the component and keeping all external edges. *Condensation* is the process of doing this for all components.

The structure of strong components

Theorem: The condensation of every directed graph is a DAG

Sketch of proof by contradiction:

- A DAG is simply a directed graph with no cycles
- The alternative to the theorem is that the condensation contains a cycle
- But then the vertices (in the original graph) of the components involved in the cycle would be mutually reachable
- Therefore they would form a larger component than we started with

Implications:

Every directed graph can be seen at two scales:

- Micro: every component is composed of one or more “tangled” cycles (in the original graph)
- Macro: the “meta nodes” form a DAG.
- So inside a component, you can loop around as long as you want.
- But every is always going “downhill”. Take one, and you can never go back to the component you just left.

Strong components and topological sorting

If C is a strong component, let F(C) be the highest post-visit number among its vertices.
L*emma 2*: If the condensation DAG has an edge A->B, then, after running DFS, F(A)>F(B)

Proof: there are two possibilities:

a) We visit a vertex in B before any vertex in A. Since the condensation is a DAG and there is an edge A->B, A must not be reachable from B (otherwise they'd be in the same component). So the DFS call in B will not discover any vertices in A. DFS will later restart from some vertex that does reach A. F(A)>F(B) holds

b) We visit a vertex in A before any vertex in B. In this case, B is reachable from A, so DFS will discover B while A is still on the stack. All vertices reachable from B will be discovered and receive a post number before we finish examining the neighbors of the vertex that first discovered B. So F(A)>F(B) holds.

Corollary

- The component that holds the maximum post number is a source
- This property is maintained if we iteratively remove the component with

Therefore, we can topologically sort G with a single DFS pass.

Computing Strong Components - Kosajaru's algorithm

We now have all the tools we need. Let G be a directed graph and G' its condensation

- G' is a DAG. Therefore:
  - It has at least one **sink** and one **source** (lemma 1).
  - It can be topologically sorted, and the highest post-number in G is in a **source** node in G' (lemma 2)

- We are very close to computing components:
  - If we happen to run DFS from a **sink**, we are guaranteed to discover exactly the vertices in that component. This suggest the following algorithm:
    - While there are any vertices left:
      - Identify a vertex that is in a sink.
      - Run DFS from that vertex.
      - Remove all discovered vertices. They form a strong component.
  - Lemma 2 gives us a way to identify a vertex in a **source** component. But we need a **sink.**
    - How can we turn a **source** into a **sink?**

Computing Strong Components - Kosajaru's algorithm

- How can we turn a **source** into a **sink?**
  - Flip all edges!
  - The components in the flipped graph are the same as in the original

We can therefore compute all components with two DFS passes:
Pass 1 (Visit): run DFS normally, (using the out- adjacency), calculating post-visit numbers
Pass 2 (Assign): Run DFS on the flipped graph (or using the in- adjacency), in reverse post-visit order. Every restart corresponds to a component

One implementation detail: we don't want to *sort* vertices by post visit (this would be O(n log n)), we can use a stack to store vertices in reverse post order instead.